



Workpath SDK Config Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1 Introduction 3

1.1 Document conventions3

2 Config Sample on a device 4

3 Life cycle of an application configuration..... 5

4 Config Sample Flow 6

5 Initializing SDK..... 7

6 Checking ConfigService Capability 9

7 Handling application configuration 10

8 Monitoring a config change event 11

9 Various ways to use configuration. 13

10 Legal notice 16

11 Support..... 17

1 Introduction

Config Sample provides use case scenario how to read and update application's data which is stored on a device using ConfigService of the HP Workpath SDK API.

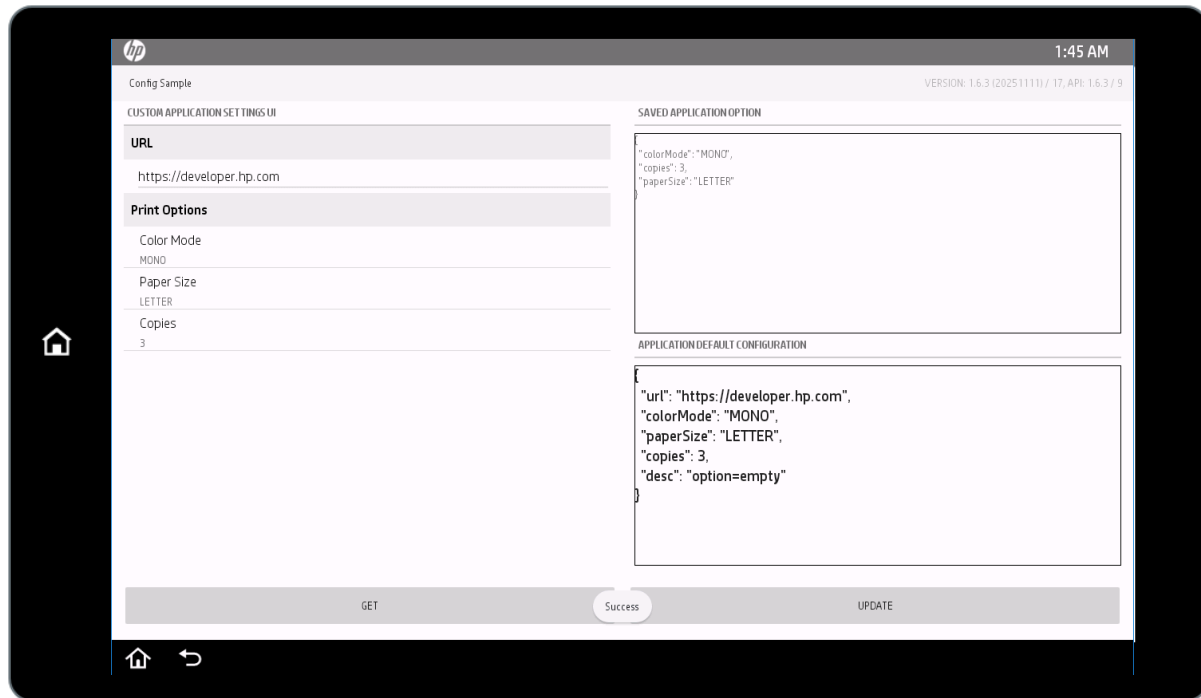
1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 Config Sample on a device

Config Sample shows the code and the usage of example how to use ConfigService when workpath application manages its own configuration. The Workpath Platform allows 64KB to an application for managing application's configuration.



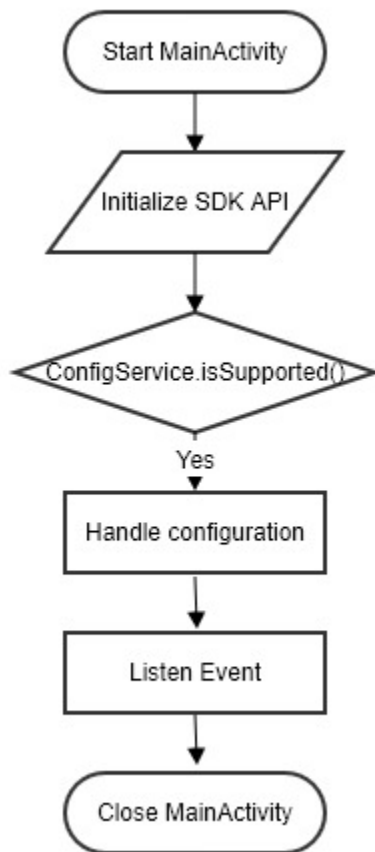
A main activity of Config Sample

3 Life cycle of an application configuration

The Workpath Platform provides configuration management system for configuration information that application uses.

Configuration is created when application is installed on a device by Workpath Platform. It's deleted when application is uninstalled.

4 Config Sample Flow



Flow of Config Sample

5 Initializing SDK

Application requires initializing the SDK services when starts with an activity for accessing the SDK services. The `com.hp.workpath.api.Workpath` includes `initialize()` method for this.

The sample shows how to initialize SDK.

1. Call `initialize()` method of Workpath SDK on resume event, and clear it on pause event. It is for calling initialization when an activity is switched. Refer to `MainActivity.java`.

```

1. @Override
2. protected void onResume() {
3.     super.onResume();
4.
5.     // register ConfigChangeObserver to receive config change callback
6.     mConfigChangeObserver.register(getApplicationContext());
7.
8.     mInitializationTask = new InitializationTask(this);
9.     mInitializationTask.execute();
10. }
11.
12. @Override
13. protected void onPause() {
14.     super.onPause();
15.
16.     // unregister ConfigChangeObserver
17.     mConfigChangeObserver.unregister(getApplicationContext());
18.
19.     mInitializationTask.cancel(true);
20.     mInitializationTask = null;
21.
22.     if (mConfigUpdateTask != null) {
23.         mConfigUpdateTask.cancel(true);
24.         mConfigUpdateTask = null;
25.     }
26.
27.     ...
28. }

```

Initialization call in Activity Resume

2. Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, sdk exception or security exception will be thrown. Refer to `InitializationTask.java`.

```

1. @Override
2. protected InitStatus doInBackground(final Void... params) {
3.     InitStatus status = InitStatus.NO_ERROR;
4.
5.     try {
6.         // initialize Workpath SDK
7.         Workpath.getInstance().initialize(mContextRef.get());
8.     } catch (SdkUnsupportedException sue) {
9.         Log.e(TAG, "SDK is not supported!", sue);
10.        mException = sue;
11.        status = InitStatus.INIT_EXCEPTION;
12.    } catch (SecurityException se) {
13.        Log.e(TAG, "Security exception!", se);
14.        mException = se;
15.        status = InitStatus.INIT_EXCEPTION;
16.    }
17.
18.    // Check if ConfigService is supported
19.    if (status == InitStatus.NO_ERROR
20.        && !ConfigService.isSupported(mContextRef.get())) {
21.        // ConfigService is not supported on this device
22.        status = InitStatus.NOT_SUPPORTED;
23.    }
24.
25.    return status;
26. }

```

Exception handling by initializationTask

6 Checking ConfigService Capability

The `isSupported()` method returns whether the ConfigService is supported. Refer to `InitializationTask.java`.

```
1. // Check if ConfigService is supported
2. if (status == InitStatus.NO_ERROR
3.     && !ConfigService.isSupported(mContextRef.get())) {
4.     // ConfigService is not supported on this device
5.     status = InitStatus.NOT_SUPPORTED;
6. }
```

Available service check by `isSupported()`

7 Handling application configuration

ConfigService provides methods for getting and setting a default configuration.

The code snippet shows example of such queries :

```

1. @Override
2. protected JSONObject doInBackground(Void... voids) {
3.     // call ConfigService API to retrieve configuration for the application
4.     JSONObject configuration = ConfigService.getDefaultConfig(mContextRef.get().getApplic
        icationContext(), result);
5.
6.     // check Result first
7.     if (result.getCode() == Result.RESULT_OK) {
8.         return configuration;
9.     }
10.    return null;
11. }

```

Getting default config for an application in ConfigReaderTask.java

```

1. @Override
2. protected Result doInBackground(final Void... voids) {
3.     Result result = new Result();
4.
5.     if (mContextRef.get() != null) {
6.         try {
7.             // convert string value to json object
8.             JSONObject json = new JSONObject(value);
9.
10.            // call ConfigService API to set new configuration value for current applic
                ation
11.            result = ConfigService.setDefaultConfig(mContextRef.get(), json);
12.        } catch (JSONException e) {
13.            result = new Result(Result.RESULT_FAIL, Result.ErrorCode.NOT_SUPPORTED, "No
                t a valid JSON value");
14.        }
15.    }
16.
17.    return result;
18. }

```

Updating default config for an application in ConfigUpdateTask.java

8 Monitoring a config change event

When application's config is changed, the Workpath SDK will send an event. The application can detect the change through `onChange()` event.

The code snippet shows example of such queries :

1. Register observer to get an event.

```

1. @Override
2. protected void onResume() {
3.     super.onResume();
4.
5.     // register ConfigChangeObserver to receive config change callback
6.     mConfigChangeObserver.register(getApplicationContext());
7.
8.     mInitializationTask = new InitializationTask(this);
9.     mInitializationTask.execute();
10. }

```

Registering an event in MainActivity.java

2. Extend `ConfigService.AbstractConfigChangeObserver` to detect an event.

```

1. /**
2.  * Receives notification about config update.
3.  */
4. private class ConfigChangeObserver extends ConfigService.AbstractConfigChangeObserver {

5.     public ConfigChangeObserver(Handler handler) {
6.         super(handler);
7.     }

8.
9.     @Override
10.    public void onChange(JSONObject updatedData) {
11.        Log.d(TAG, "Received onChange");
12.        showSnackBar("Configuration is changed: " + updatedData);
13.
14.        mAlertDialog = new AlertDialog.Builder(MainActivity.this)
15.            .setTitle(getString(R.string.config_observer))
16.            .setMessage(getString(R.string.config_change))
17.            .setCancelable(false)
18.            .setPositiveButton(R.string.ok, new DialogInterface.OnClickListener() {

19.                @Override
20.                public void onClick(DialogInterface dialog, int which) {
21.                    new ConfigReaderTask(MainActivity.this, true).execute();
22.                    dialog.dismiss();
23.                }
24.            })
25.            .setNegativeButton(R.string.cancel, new DialogInterface.OnClickListener() {

26.                @Override
27.                public void onClick(DialogInterface dialog, int which) {
28.                    new ConfigReaderTask(MainActivity.this, false).execute();

```

```
29.             dialog.dismiss();
30.         }
31.     })
32.     .show();
33. }
34. }
35. Extending an event in MainActivity.java
36.
37. Unregister observer.
38. @Override
39. protected void onPause() {
40.     super.onPause();
41.
42.     // unregister ConfigChangeObserver
43.     mConfigChangeObserver.unregister(getApplicationContext());
44.
45.     ...
46. }
```

Unregistering an event in MainActivity.java

9 Various ways to use configuration

Through configuration, various application behaviors can be achieved.
Here is an example of one:

Requirement

- Secret values should be stored securely in local application sandbox storage.
- Secret value should not remain in configuration data.

For the above requirements, the scenario can be designed as follows:

Scenario

- When configuration is updated, the updated config data is received through an observer.
- If the data contains the item "secret," its value is extracted.
- The extracted data is encrypted and stored in the internal storage.
- The configuration data with the "secret" item is updated by removing the "secret" data.

Implementation details for the scenario:

1. Receiving event from the ConfigurationChangeObserver

```

1. private class ConfigChangeObserver extends ConfigService.AbstractConfigChangeObserver {
2.     public ConfigChangeObserver(Handler handler) {
3.         super(handler);
4.     }
5.
6.     @Override
7.     public void onChange(JSONObject updatedData) {
8.         if (!isSecretValueUpdate) {
9.             showSnackBar("ConfigService onChange()");
10.            mAlertDialog = new AlertDialog.Builder(MainActivity.this)
11.                .setTitle(getString(R.string.config_observer))
12.                .setMessage(getString(R.string.config_change))
13.                .setCancelable(false)
14.                .setPositiveButton(R.string.ok, new DialogInterface.OnClickListener() {
15.                    @Override
16.                    public void onClick(DialogInterface dialog, int which) {
17.                        getConfigComplete(updatedData);
18.                        updatePrintOption(updatedData);
19.                        dialog.dismiss();
20.                    }
21.                })
22.                .setNegativeButton(R.string.cancel, new
23.                DialogInterface.OnClickListener() {
24.                    @Override
25.                    public void onClick(DialogInterface dialog, int which) {
26.                        getConfigComplete(updatedData);
27.                        dialog.dismiss();
28.                    }
29.                })
30.                .show();
31.        } else {
32.            isSecretValueUpdate = false;
33.            getConfigComplete(updatedData);
34.        }
35.    }

```

2. Checking "secret" key and value

If the "secret" key exists, update the Configuration data by removing the "secret" item.

```

1. private boolean isExistSecretComponent(JSONObject jsonObject) {
2.     /**
3.      * This source code is one of example that how configuration can be used various way.
4.      */
5.     if (jsonObject.has(SECRET_KEY)) {
6.         try {
7.             String secretValue = jsonObject.getString(SECRET_KEY);
8.             jsonObject.remove(SECRET_KEY);
9.
10.            mConfigUpdateTask = new ConfigUpdateTask(MainActivity.this,
11.            jsonObject.toString());
12.            mConfigUpdateTask.taskExecute();
13.            saveSecretValue(secretValue);
14.            Logger.showResult(MainActivity.this, getString(R.string.secret_value));
15.            return true;
16.        } catch (JSONException e) {
17.            if (getApplicationContext() != null) {
18.                Logger.showResult(MainActivity.this, getString(R.string.not_supported,
19.                SECRET_KEY) + " " + e.getMessage());
20.            }
21.        }
22.        return false;
23.    }
24.

```

3. Storing and retrieving secret values:

Encrypt secret values and store them in the internal application sandbox storage.

```

1. private void saveSecretValue(String secretValue) {
2.     try {
3.         KeyGenParameterSpec keyGenParameterSpec = MasterKeys.AES256_GCM_SPEC;
4.         String mainKeyAlias = MasterKeys.getOrCreate(keyGenParameterSpec);
5.
6.         File secretFile = new File(getFilesDir(), SECRET_FILE);
7.         if (secretFile.exists()) {
8.             secretFile.delete();
9.         }
10.
11.         EncryptedFile encryptedFile = new EncryptedFile.Builder(
12.             secretFile,
13.             MainActivity.this,
14.             mainKeyAlias,
15.             EncryptedFile.FileEncryptionScheme.AES256_GCM_HKDF_4KB
16.         ).build();
17.
18.         byte[] fileContent = secretValue.getBytes(StandardCharsets.UTF_8);
19.         OutputStream outputStream = encryptedFile.openFileOutput();
20.         outputStream.write(fileContent);
21.         outputStream.flush();
22.         outputStream.close();
23.     } catch (GeneralSecurityException | IOException e) {
24.         if (getApplicationContext() != null) {
25.             Logger.showResult(MainActivity.this, e.getMessage());
26.         }
27.     }
28.

```

```
29.
30.     private String loadSecretValue() {
31.         try {
32.             KeyGenParameterSpec keyGenParameterSpec = MasterKeys.AES256_GCM_SPEC;
33.             String mainKeyAlias = MasterKeys.getOrCreate(keyGenParameterSpec);
34.
35.             File file = new File(getFilesDir(), SECRET_FILE);
36.
37.             if (!file.exists())
38.                 return null;
39.
40.             EncryptedFile encryptedFile = new EncryptedFile.Builder(
41.                 file,
42.                 MainActivity.this,
43.                 mainKeyAlias,
44.                 EncryptedFile.FileEncryptionScheme.AES256_GCM_HKDF_4KB
45.             ).build();
46.
47.             InputStream inputStream = encryptedFile.openFileInput();
48.             ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();
49.             int nextByte = inputStream.read();
50.             while (nextByte != -1) {
51.                 byteArrayOutputStream.write(nextByte);
52.                 nextByte = inputStream.read();
53.             }
54.
55.             return byteArrayOutputStream.toString();
56.         } catch (GeneralSecurityException | IOException e) {
57.             if (getApplicationContext() != null) {
58.                 Logger.showResult(MainActivity.this, e.getMessage());
59.             }
60.         }
61.         return null;
62.     }
63.
```

The complete implementation of the scenario can be found in ConfigSample.

10 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

11 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.